

Jão Vitor Levorato - Séfora Davanso - Thiago Cristovão

Classboard



Objetivo



- **Objetivo Geral:** Desenvolver uma plataforma web completa para gestão e análise de dados educacionais
- **Objetivos Específicos:**
 - Implementar sistema de multi-tenancy com isolamento de dados por usuário
 - Criar dashboard interativo com gráficos e relatórios
 - Desenvolver API REST com FastAPI e autenticação JWT
 - Integrar frontend React com backend Python
 - Permitir upload de dados e análise com IA

Modelagem do Banco de Dados

- Diagrama ER mostrando as tabelas principais:
 - users: sistema de usuários
 - alunos: com user_id para multi-tenancy
 - avaliacoes: provas, trabalhos
 - notas: relacionamento aluno-avaliacao
 - faltas: controle de frequência
 - turmas: agrupamento de alunos
- Relacionamentos: Foreign Keys e cardinalidades
- Implementação: SQLite com SQLAlchemy ORM



Configuração do banco e notas

1. Código para configuração do banco e testes de resetando opções para resetar ou debugar o status do banco
2. Simplificação dos códigos de filtro melhorando sua legibilidade e usabilidade

```
def filter_notas(db: Session, current_user: User, **criteria) -> List[Nota]:  
    query = db.query(NotaTable).join(AvaliacaoTable)  
  
    # Se não for admin, filtrar apenas pelas notas do usuário  
    if should_filter_by_user(current_user):  
        user_id = get_user_id_from_db(current_user.username, db)  
        query = query.filter(NotaTable.user_id == user_id)  
  
    if criteria.get('aluno_id'):  
        query = query.filter(NotaTable.aluno_id == criteria['aluno_id'])  
    if criteria.get('materia'):  
        query = query.filter(AvaliacaoTable.materia == criteria['materia'])  
    if criteria.get('tipo'):  
        query = query.filter(AvaliacaoTable.tipo == criteria['tipo'])  
    if criteria.get('bimestre'):  
        query = query.filter(AvaliacaoTable.bimestre == criteria['bimestre'])  
  
    notas = query.all()
```



Sistema de Autenticação e Multi-Tenancy

1. Autentica usuários com JWT tokens seguros
2. Isola dados por usuário - cada usuário só vê seus próprios dados
3. Administrador especial que vê todos os dados
4. Controla acesso a todos os endpoints da API

```
def get_alunos(db: Session = Depends(get_db),  
              current_user: User = Depends(get_current_user)):  
    query = db.query(AlunoTable)  
    if should_filter_by_user(current_user):  
        user_id = get_user_id_from_db(current_user.username, db)  
        query = query.filter(AlunoTable.user_id == user_id)  
    return query.all()
```



Sistema de Relatórios e Análises Complexas



1. Calcula estatísticas em tempo real, como médias, rankings e aprovações
2. Gera relatórios com dados agregados por matéria/bimestre/turma
3. Cria rankings de alunos por desempenho
4. Analisa correlações entre matérias e desempenho

```
@router.get("/turma/{turma_id}/estatisticas")
def get_turma_stats(turma_id: int, db: Session = Depends(get_db)):
    # Query complexa com JOINS múltiplos
    stats = db.query(NotaTable, AvaliacaoTable, AlunoTable)\
        .join(AvaliacaoTable)\
        .join(AlunoTable)\
        .filter(TurmaTable.id == turma_id)\
        .group_by(AvaliacaoTable.materia)\
        .all()
```


Obrigado!

